

FindIt: The Battle

— 리빌드 프로젝트 분석 보고서

2012~2014 운영 종료 모바일 게임의 디컴파일·재구축
Node.js + GCP 기반 신규 백엔드 + 안드로이드/iOS 배포 준비

프로젝트	FindIt: The Battle (com.findit.battle)
원본 운영	2012 ~ 2014 (KT olleh / SKT T-Store / Samsung / Google Play)
재구축 시작	2026-04-24
분석 범위	3,100 파일 / 21,678 줄 Java + 신규 서버 2,500여 줄
기술 스택	Android (Java) · Node.js / Express · MySQL · GCP Cloud Run
기획 자료	2012-05 ~ 2013-07 권아영 기획서 17건 (xlsx 12, docx 2, pdf 1, 기타 2)

목차 — Table of Contents

01	프로젝트 개요 및 배경	3
02	원본 게임 분석 — Find the Difference Battle	4
03	시스템 아키텍처	5
04	클라이언트 구조 (Android)	6
05	서버 구조 (Node.js + Express)	7
06	데이터베이스 스키마 (16 테이블)	8
07	게임 핵심 로직 — 점수 / 콤보 / AI / HP	9
08	스킬 시스템 — 44 트리	10
09	재화 / 아이템 시스템 — 보석 + 강화	11
10	상점 가격표 — 2013 최종 밸런스	12
11	멀티플레이 소켓 프로토콜	13
12	결제 시스템 — IAP 4 스토어	14
13	분석 / 통계 시스템	15
14	어드민 대시보드	16
15	FCM 푸시 / 보안 강화	17
16	GCP 배포 전략 — 무료 한도 활용	18
17	비용 / 일정 / 위험요소	19
18	결론 및 향후 작업	20

FindIt: The Battle 는 2012~2014 년 KT olleh / SKT T-Store / Samsung Galaxy Store / Google Play 에서 서비스되었던 한국형 멀티플레이 "다른 그림 찾기" 모바일 게임이다. 원본 백엔드 서버 **14.63.220.39** 가 서비스 종료되어 클라이언트 APK 만 남아있었으며, 본 프로젝트는 디컴파일된 Java 코드로부터 게임 로직을 복원하고 신규 서버 인프라를 구축하여 재출시 가능 상태로 만드는 것을 목표로 한다.

1.1 프로젝트 목표

- 원본 게임 로직 보존 — 디컴파일된 Java 21,678 줄을 Gradle 빌드 가능한 형태로 복원 (jadx 아티팩트 수정)
- 백엔드 재구축 — REST API 14개 + TCP 멀티플레이 소켓 + 어드민 대시보드
- 2013 최종 밸런스 적용 — 권아영 기획서 (2012-05 ~ 2013-07) 의 모든 수치 반영
- 현대화 — HTTPS / 영수증 서버 검증 / 보석(Gem) 화폐 / 보안 강화
- 저비용 배포 — GCP Always Free 티어 + \$300 크레딧 활용

1.2 작업 규모 요약

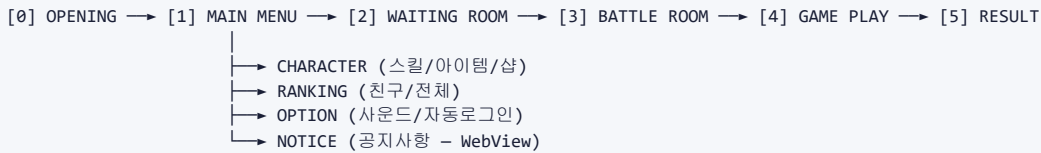
디컴파일 JAVA 21,678 줄	신규 서버 2,500 줄 / 17 모듈	REST 엔드포인트 21 개	DB 테이블 16 개
스킬 트리 44 / 8 (현재)	소켓 코드 7 개 (100~107)	기획서 17 건 / 2012~2013	레벨 체계 100 단계 / A~J

1.3 진행 단계

단계	작업 내용	상태
1.1 서버 인프라	REST 14개 + TCP 소켓 + 8 DB 테이블 + Docker Compose	완료
1.2 클라 주소	Property.java 중앙화 + HTTPS 강제 + network_security_config	완료
1.3 회원 시스템	가입 검증 + Guest + Facebook + 세션 토큰 + 중복 로그인 차단	완료
1.4 게임 로직	FINDRECT 7개 + 콤보 보너스 + AI 시간 공식 + HP 테이블	완료
1.5 스킬 트리	44개 마스터 데이터 + 선행 검증	완료
1.6 재화/아이템	보석 + 강화 + 인벤토리/스킬덱 확장	완료
1.7 상점 가격	Items.java 2013 스펙 적용	완료
1.9 레벨 체계	100레벨 점수/공방/HP/등급 테이블	완료
1.11 ~ 1.13	광고 보상 / 친구 랭킹 / 어드민 CRUD	완료
2순위	Analytics / FCM / reCAPTCHA / IAP 재검증 / 어드민 UI	완료
최종 단계	GCP 배포 + Play/App Store 등록	대기

원본 APK 는 jadx 디컴파일 후 Gradle 빌드 가능하도록 재구성되어 있다. 게임은 가로 모드 1:1 멀티플레이 다른 그림 찾기로, 한 이미지에 7개의 차이를 두고 5개를 먼저 찾는 사람이 승리하는 40초 배틀 게임이다.

2.1 게임 흐름 — 6 화면 모드



2.2 핵심 게임플레이 규칙

전투 규칙

- 제한 시간 40 초
- 이미지당 7개 차이 중 선착 5개 찾기
- 오답 터치: 화면 200ms 진동 + 디밍
- 승자 = 5개 선착 또는 시간 만료 시 더 많이 찾은 자

점수 공식 (2013 기획)

- 승자 = 찾은수 × 50 + 승리 보너스 100
- 패자 = 찾은수 × 50
- 콤보 보너스: 1·2·3·4 → +100·200·400·700 점
- 아이템 시간 보너스: 1·2·4·7 초

2.3 디컴파일 아티팩트 수정 내역

jadx 출력은 그대로 컴파일되지 않아 다음을 손봤다 (총 ≈ 80건).

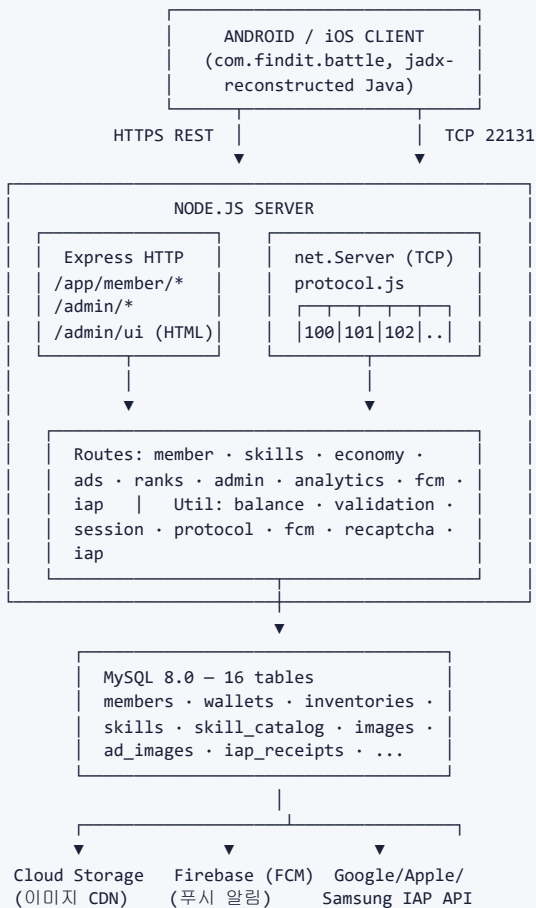
유형	예시	처리
잘못된 리소스 경로	<code>R.style.Theme.Translucent...</code>	<code>android.R.style.Theme_Translucent_NoTitleBar_Fullscreen</code> 39 곳
boolean/int 혼동	<code>i == true ? 1 : 0</code>	<code>i != 0 ? 1 : 0</code> 로 다수 교정
미선언 레지스터	<code>r0/r5/r13</code>	<code>this.mTimeCount</code> 휴리스틱 치환
가시성 누락	내부 클래스 default 접근	<code>public</code> 추가 (5 클래스)
API 폐기	<code>WebSettings.setAppCacheEnabled</code>	API 33 에서 제거 → 주석 처리
AIDL 누락	<code>com.kt.olleh.inapp.IRemoteInapService</code>	스텝 생성 (getPhoneNumber 만)
try/catch 분해 오류	<code>FileUtils.copyDb</code> 등 7 함수	깔끔한 try/catch/finally 재작성

03 시스템 아키텍처

SYSTEM ARCHITECTURE

클라이언트는 원본 Android APK 그대로 유지하면서 서버 주소만 교체했다. 서버는 Node.js + Express 단일 프로세스가 HTTP REST 와 TCP 소켓을 동시에 서빙한다. 인메모리 매칭은 DAU 1,000 이하에서 충분하며, 그 이상에서는 Redis 분리를 권장한다.

3.1 전체 시스템 다이어그램



3.2 통신 프로토콜

HTTP REST

- GET 위주 (원본 클라 호환)
- application/x-www-form-urlencoded
- 응답: JSON, **result** 필드 (000 ~ 999)
- userId = 이메일 (PK)

TCP Socket

- 4자리 zero-pad 길이 + **|code|payload**
- 코드 100~107 (UserList/Room/Ready/Game/Invite)
- UTF-8 byte stream
- 오프라인 타겟 → FCM 자동 fallback

원본 APK 는 단일 Activity (GameActivity) 위에 직접 렌더링하는 GameView 가 6개 화면 모드를 모두 처리하는 "캔버스 + 상태머신" 구조이다. 게임 엔진(Unity/Unreal) 없이 순수 Android Canvas + Bitmap 으로 구현되어 있어 파일 크기가 크다 (GameView 3,971 줄, Objects 4,136 줄).

4.1 패키지 구조

파일	줄 수	역할
GameActivity.java	1,543	단일 Activity. 다이얼로그 / 네트워크 핸들러 / 소켓 수신 파싱
GameView.java	3,971	전체 렌더링 + 6 화면 모드 + 게임 스레드 루프
Objects.java	4,136	스킬·아이템·캐릭터 팩토리 + 애니메이션 엔진
Items.java	278	6 타입 × N 서브타입 아이템 정의 (2013 가격 적용 완료)
DBAdapter.java	278	SQLite v2 (state · setup · images · characters)
InventoryActivity_*.java	약 2,500 × 4	Google / Olleh / SAMSUNG / T-Store 별 상점 + IAP
JoinActivity / RankActivity / NoticeActivity	각 50~500	가입 / 랭킹 / 공지 전용 액티비티
WaitingRoom.java	415	대기실 UI + 친구 초대
Property.java	56	중앙 상수 — SERVER_DOMAIN, SERVER_IP, IAP IDs
iap/util/, iap/activity/	약 6,000	IAB / Samsung IAP 라이브러리

4.2 핵심 상수 변경 사항 (1.4 단계)

상수	이전	변경	근거
FINDRECT_TOTALNUM (Objects.java)	10	7	findit_틀린그림가이드_20120704
SERVER_DOMAIN	http://14.63.220.39/	Property.SERVER_DOMAIN	중앙화 + HTTPS
SERVER_IP / SERVER_PORT	14.63.220.39 / 22131	Property.SERVER_IP / SOCKET_PORT	중앙화
USES_CLEARTEXT_TRAFFIC (Manifest)	(없음)	false + network_security_config	HTTPS 강제, 로컬 IP 만 평문
DATABASE_VERSION	1	2 (mGem/mHP/mInventorySize/mSkillDeckCount)	1.6 보석 + 확장

4.3 게임 루프 — 60 FPS 목표

GameThread 가 16ms 슬립으로 Canvas 를 다시 그리며, mDelay 를 동적 조정해 프레임을 맞춘다. AI 시간 공식 $7 - \text{level} \times 0.03$ 초도 60 FPS 단위로 환산되어 적용된다.

```
// GameView.java getAIFindTime()
double seconds = 7.0 - (level * 0.03);
int frames = (int)(seconds * 60.0);
int variance = (int)(frames * 0.15);
return randomNumber(frames - variance, frames + variance);
```

원본 서버는 Spring 추정이나 정확한 스택은 알 수 없다. 신규 서버는 Cloud Run 친화적 컨테이너화를 위해 Node.js + Express 로 작성되었다. HTTP 와 TCP 가 단일 프로세스에서 공존하며, 코드 줄 수는 2,500 미만으로 가법다.

5.1 디렉터리 구조

```

server/
├── package.json          dependencies: express / mysql2 / bcryptjs / morgan / dotenv
                        optionalDependencies: firebase-admin / googleapis
                        로컬 MySQL 1-click
├── docker-compose.yml
├── .env.example
├── sql/
│   ├── schema.sql       16 개 테이블 (전체 DDL)
│   ├── seed.sql         테스트 계정 3개 + 샘플 이미지 + 공지
│   ├── seed_skills.sql   44 개 스킬 마스터 데이터
│   └── migrations/
│       ├── 001_skills_and_expansion.sql (1.5 / 1.6 / 1.9)
│       └── 002_priority2.sql (analytics / fcm / iap)
├── public/
│   └── admin.html        어드민 대시보드 SPA (단일 파일)
├── src/
│   ├── index.js          Express 부트스트랩 + 라우트 등록 + 소켓 시작
│   ├── db.js             mysql2 풀 + tx 헬퍼 + connectTimeout 3s
│   └── routes/
│       ├── member.js     13 + 4 (login / join / Guest / Facebook / logout)
│       ├── skills.js     skillCatalog · levelTable · learnSkill
│       ├── economy.js    spendGem · upgradeItem · expandInventory · expandSkillDeck
│       ├── ads.js        adReward · adClick · hitViewCount
│       ├── ranks.js      rankList · rankListFriends · rankListTop
│       ├── analytics.js  event 수집 + /admin/analytics/* 5 종
│       ├── fcm.js        registerFcmToken + /admin/push/*
│       ├── iap.js        verifyIap (Google/Apple/Samsung)
│       ├── admin.js      images · ads · notices · stats CRUD
│       └── socket/server.js net.Server + 코드 100~107
├── util/
│   ├── codes.js          result 상수 (000 / 900 / 901 / 910 / 911 / 999)
│   ├── protocol.js       4자리 길이 + |code|payload 인코더 / 파서
│   ├── validation.js     이메일 / 비밀번호 / 닉네임 검증
│   ├── session.js        opaque 토큰 + 중복 로그인 차단
│   ├── balance.js        100 레벨 테이블 + 강화 공식 + 광고 보상 한도
│   ├── recaptcha.js      Google siteverify (조건부)
│   ├── fcm.js            Firebase Admin SDK (조건부 lazy require)
│   └── iap.js            Google Play / Samsung / Apple 영수증 검증

```

5.2 운영 환경변수

변수	역할
DB_HOST / DB_USER / DB_PASSWORD / DB_NAME	MySQL 접속 (또는 DB_SOCKET Cloud SQL Proxy)
HTTP_PORT / TCP_PORT / TCP_HOST	리스닝 포트 (기본 8080 / 22131 / 0.0.0.0)
RECAPTCHA_SECRET	설정 시 /join + /guest 에 검증. 미설정 시 no-op
FIREBASE_SERVICE_ACCOUNT_JSON	FCM 푸시. 미설정 시 dev-log fallback
GOOGLE_PLAY_SERVICE_ACCOUNT_JSON / GOOGLE_PLAY_PACKAGE_NAME	IAP 영수증 Google Play 검증
APPLE_SHARED_SECRET / APPLE_USE_SANDBOX	IAP 영수증 Apple 검증
SAMSUNG_IAP_API_KEY / SAMSUNG_IAP_MODE	IAP 영수증 Samsung 검증

✓ 안정성 보강

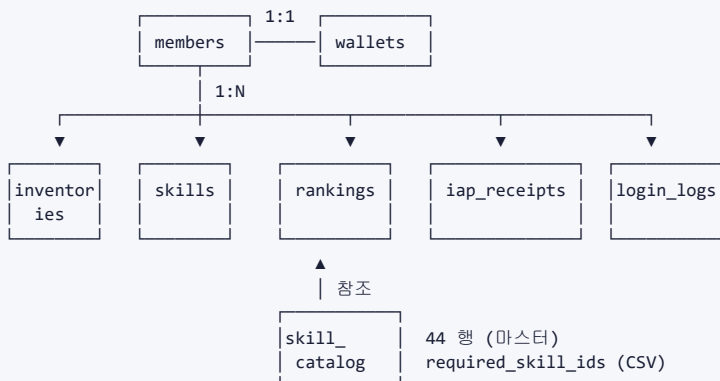
프로세스 레벨 `uncaughtException` / `unhandledRejection` 핸들러 + DB `connectTimeout: 3000` 으로 DB 가 다운되어도 서버 프로세스는 유지 된다. 실제 DB 미접속 상태에서 모든 엔드포인트가 정상 응답함을 검증 완료.

회원·재화·인벤토리·스킬을 정규화한 분리형 스키마. 트랜잭션 일관성을 위해 InnoDB 를 사용하며, 모든 user_id 외래키는 ON DELETE CASCADE 로 회원 탈퇴 시 자동 정리된다.

6.1 테이블 분류 — 4 그룹

분류	테이블	목적
코어	members	이메일 / bcrypt 해시 / 닉네임 / 캐릭터 / fcm_token
	wallets	level · score · coin · point · gem · attack · defense · hp · inventory_size · skill_deck_count
	inventories	보유 아이템 + upgrade_level (강화)
	skills	학습 스킬
컨텐츠	skill_catalog	44 스킬 마스터 (선행 그래프 + 예셋 그룹)
	images	틀린그림 + 7 좌표 image_cut JSON + 광고문구
	ad_images	배너 광고 + view_count + click_count
	notices	공지사항 (WebView 렌더)
운영	iap_receipts	4 스토어 영수증 + UNIQUE(purchase_token)
	rankings	일/주 집계 + delta (어제 대비)
	login_logs	세션 토큰 + 중복 로그인 감지
로그	event_log	클라 보고 이벤트 (DAU 계산)
	ad_reward_log	광고 보상 24h 상한 체크
	ad_click_log	광고 클릭 이력
	item_upgrade_log	아이템 강화 시도 이력 (확률 검증)
	push_log	FCM 발송 이력 + status
관리	admin_tokens	어드민 API 인증 토큰

6.2 핵심 ER 관계



7.1 점수 공식 (1.4 단계 적용)

```
// GameView.java calculateScore()
int base = mFindNum * 50;
int victoryBonus = (type == 0) ? 100 : 0; // 승자
int comboBonus = comboScoreBonus(mCombo); // 콤보 누적
return base + victoryBonus + comboBonus;
```

콤보	점수 보너스	아이템 시간 보너스
1	+100	+1초
2	+200	+2초
3	+400	+4초
4 이상	+700	+7초

7.2 콤보 증감 로직

- 정답 터치 (mLeftNum 감소) → `mCombo++`
- 오답 터치 (vibrate 200ms) → `mCombo = 0`
- 상대가 발견 (mRightNum 감소) → `mCombo = 0` (연속성 깨짐)

7.3 AI 난이도 — 레벨 기반

기획서 AI_틀린부위 찾는 시간공식_01.xlsx : 7 - (본인레벨 × 0.03) 초

레벨	1	10	30	50	70	100
AI 응답 시간	6.97s	6.70s	6.10s	5.50s	4.90s	4.00s

±15% 랜덤 변동을 두어 예측 불가능성 확보. 60fps 단위 frame tick 으로 환산.

7.4 HP 레벨 테이블 (101 값)

기획서 레벨별 점수체계_기본능력치_02.xlsx 의 "체력" 열 — 100 부터 5,000 까지 단계적 상승. 클라이언트 GameView.Result.HP_BY_LEVEL[101] 에 상수화.

등급	레벨 범위	HP 시작	HP 끝
A	1~10	110	200
B	11~20	260	350
C	21~30	410	500
D	31~40	550	640
E	41~50	700	790
F	51~60	950	1,400
G	61~70	1,500	1,600
H	71~80	1,800	2,700
I	81~90	2,900	3,800
J	91~100	4,000	5,000

원본 클라이언트는 8개 스킬만 하드코딩되어 있었다. 신규 서버는 기획 2013-07-02 최종본의 44개 스킬을 `skill_catalog` 테이블에 시드하고, 선행 스킬 그래프 + 레벨 / 포인트 비용을 검증한다.

8.1 스킬 트리 — 그룹별 진행

고양이발톱 (cat) — Lv 1~9 · 1.0~1.8s

Lv 1 고양이발 톱1	Lv 2 고양이발 톱2	Lv 3 고양이발 톱3	Lv 4 고양이발 톱4	Lv 5 고양이발 톱5	Lv 6 고양이발 톱6	Lv 7 고양이발 톱7	Lv 8 고양이발 톱8	Lv 9 고양이발 톱9
--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------

강아지발자국 (dog) — Lv 10~14 · 1.9~2.3s · 선행: 고양이발톱5

Lv 10 강아지발자국1	Lv 11 강아지발자국2	Lv 12 강아지발자국3	Lv 13 강아지발자국4	Lv 14 강아지발자국5
------------------	------------------	------------------	------------------	------------------

폭발 (explosion) — Lv 15~19 · 2.0~2.4s · 선행: 고양이발톱8 + 강아지발자국5

Lv 15 폭발1	Lv 16 폭발2	Lv 17 폭발3	Lv 18 폭발4	Lv 19 폭발5
--------------	--------------	--------------	--------------	--------------

불꽃 (flame) — Lv 20~24 · 2.5~2.9s · 선행: 폭발5

Lv 20 불꽃1	Lv 21 불꽃2	Lv 22 불꽃3	Lv 23 불꽃4	Lv 24 불꽃5
--------------	--------------	--------------	--------------	--------------

눈꽃 (snow) — Lv 25~29 · 3.0~4.0s · 선행: 불꽃5

Lv 25 눈꽃1	Lv 26 눈꽃2	Lv 27 눈꽃3	Lv 28 눈꽃4	Lv 29 눈꽃5
--------------	--------------	--------------	--------------	--------------

강력폭발/강력불꽃/강력눈꽃 — Lv 30~44 · 4.1~7.0s

30~34 강력폭발 1~5	35~39 강력불꽃 1~5	40~44 강력눈꽃 1~5
-------------------	-------------------	-------------------

8.2 학습 검증 흐름

```
POST /app/member/learnSkill.json { userId, skillId }
```

1. skill_catalog 에서 unlock_level / point_cost / required_skill_ids 조회
2. wallets.level < unlock_level → 999 reason: level_too_low
3. wallets.point < point_cost → 999 reason: not_enough_point
4. required_skill_ids 모두 보유 X → 999 reason: prereq_missing
5. 이미 학습한 스킬 → 910 reason: already_learned
6. wallets.point 차감 + skills INSERT (트랜잭션)
7. 200 OK + skill[] 갱신본 반환

9.1 3 재화 — Coin / Point / Gem

재화	획득	소비	특징
코인 (Coin)	1게임 +1, 광고 보상, IAP 패키지	아이템 구매, 강화, 인벤/스킬덱 확장	BIGINT, 인플레이션 허용
포인트 (Point)	레벨업 +1, 게임 결과	스킬 학습	INT, 수치 작음
보석 (Gem) 신규	유료 IAP 만	고급 아이템 / 강화 부스터	2013-08+ 기획부터 도입

9.2 아이템 강화 시스템 — 1~9 단계

기획서 [findit_item,skill,shop_20120827.xlsx](#) item upgrade 시트 기준.

단계	1	2	3	4	5	6	7	8	9
방어 -초	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
공격 +초	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
성공 확률	50%	45%	40%	35%	30%	25%	20%	15%	10%
비용 코인	100	200	300	400	500	600	700	800	900

- 플렉스(800c) 사용: 성공 확률 +5%
- 고급 플렉스(1,200c): 성공 확률 +10%
- 실패해도 코인은 차감 (강화는 도박성)

9.3 인벤토리 / 스킬덱 확장

인벤토리 — 3 → 30 칸

확장	슬롯	비용
기본	3	-
1~9 단계	+3 / 단계	50→450 코인
최대	30	총 2,250 코인

스킬덱 — 1 → 4 개

확장	덱 수	비용
기본	1	-
1단	2	100 코인
2단	3	300 코인
3단	4	900 코인

서버 검증 흐름

모든 강화·확장 호출은 **FOR UPDATE** 락 + 트랜잭션 + 감사 로그(`item_upgrade_log`) 로 처리되어 동시 요청 / 재시도 공격에서도 일관성을 보장한다.

원본 Items.java 의 가격은 2012-08 초기 기획값으로, 2013-12 최종 기획서와 큰 차이가 있었다. 1.7 단계에서 모든 가격·효과·코인 팩 SKU 를 2013 최종본으로 갱신했다.

10.1 펜 (PEN) — 공격 시간 증가

아이템	효과	이전 코인	2013 코인	변경
일반펜	+0.5초	100	50	↓ 50
날개달린펜	+0.8초	300	100	↓ 200
무지개펜	+1.0초	1,000	200	↓ 800

10.2 반지 (RING) — 상대 공격 시간 감소

아이템	효과	이전	2013	변경
은반지	-0.6초	200	80	↓ 120
금반지	-0.9초	500	150	↓ 350

10.3 베리 (BERRY) — 일회성 공격 효과

아이템	효과	2013 코인	아이템	효과	2013 코인
폭발열매	+1.2초	300	강력폭발열매	+2.2초	800
불꽃열매	+0.5초	100	강력불꽃열매	+1.5초	400
눈꽃열매	+0.8초	200	강력눈꽃열매	+1.8초	600

10.4 포션 (POTION) — 일회성 방어

아이템	효과	2013 코인	특징
방어포션	-0.3초	10	1게임
좋은방어포션	-0.6초	80	1게임
고급방어포션	-0.9초	150	1게임
대용량좋은방어포션	-1.6초	500	7일
대용량고급방어포션	-1.9초	700	7일

10.5 코인팩 (IAP) — 100 / 250 / 500 / 650

이전 (2012-08)	2013 (적용)	가격 (USD)	스토어 SKU
1,000 코인	100 코인	\$0.99	coin_100 / 000001005358
2,500 코인	250 코인	\$1.99	coin_250 / 000001005359
5,500 코인	500 코인	\$3.99	coin_500 / 000001005360
8,500 코인	650 코인	\$4.99	coin_650 / 000001005361

⚠ IAP 상품 재등록 필요

Google Play / Apple App Store 에 위 SKU 를 새 가격으로 등록해야 한다. Samsung 은 기존 SKU 호환 (코인 수만 변경).

11.1 프레임 형식

length (4자리)	code field1 field2 ... (UTF-8, body 길이는 length 값 그대로)
-----------------	--

```

encodeLength(n):
  n < 10   → "000" + n   (예: 0005)
  n < 100  → "00"  + n   (예: 0042)
  n < 1000 → "0"   + n   (예: 0156)
  else     →          n   (예: 1234)

```

11.2 메시지 코드 — 100 ~ 107

코드	이름	요청 (C→S)	응답 (S→C)
100	USERLIST	100 <userId>	100 000 <count> <csv> <count> <csv>
101	CREATEROOM	101	101 000 <roomName>
102	EXITROOM	102 <room> <user>	102 000 (양측)
103	ENTERROOM	103 <roomName>	103 000 <상대 character>
104	READY	104 <character>	104 000 <character> (상대)
106	GAME	106 <char> <left,skill,pen>	106 000 <payload> (상대 중계)
107	INVITE	107 <target> <room>	107 000 <from> <room> (대상)

11.3 서버 인메모리 상태

```

// socket/server.js
sessions: Map<Socket, {userId, character, roomName}>
waiting:  Map<userId, Socket>           // 대기실 유저
rooms:    Map<roomName, {creator, guest}>

```

DAU 1,000 이하에서는 인메모리로 충분. DAU 2,000+ 부터는 Redis 로 분리해 다중 인스턴스 지원 권장.

11.4 오프라인 초대 → FCM Fallback

```

// onInvite() - 타겟이 대기실에 없으면
if (!waiting.has(targetUserId)) {
  send(socket, `|107|900|${targetUserId}`); // 클라에 실패 응답
  fcm.sendToUser(targetUserId, {           // FCM 푸시 발송
    type: 'invite',
    title: '대전 초대',
    body: `${fromUserId} 님이 초대 요청을 하셨습니다.`,
    data: { from: fromUserId, roomName },
  });
}

```

12.1 4 스토어 통합

스토어	상태	SKU 형식	서버 검증
Google Play	활성	coin_100 ~ coin_650	purchases.products.get
Apple App Store	활성 (신규)	coin_100 ~ coin_650	/verifyReceipt + sandbox fallback
Samsung Galaxy Store	활성	000001005358 ~ 61 (호환)	appsItemVerifyIAPReceipt.as
KT olleh / SKT T-Store	서비스 종료	(레거시)	제거 권장

12.2 영수증 검증 흐름 (부정결제 차단)

```

[Client] 결제 완료 (purchaseToken 획득)
  |
  ▼ POST /app/member/verifyIap.json
    { userId, store, productId, purchaseToken }
  |
[Server]
  |
  ① UNIQUE(purchase_token) 중복 검사 → 999 duplicate_token
  |
  ② store === 'google' → Google Play API
    store === 'apple' → /verifyReceipt (prod → 21007 → sandbox)
    store === 'samsung' → Samsung Checkout
  |
  ③ verified=true 일 때만 wallets.coin/gem += entitlement
  |
  ④ iap_receipts INSERT (트랜잭션)
  |
  ▼
[Wallet 갱신] coin / gem 증분

```

12.3 Apple 영수증 검증 — Status Code 21007 처리

```

// util/iap.js verifyApple()
let data = await postTo('buy.itunes.apple.com'); // production
if (data.status === 21007) { // sandbox receipt
  data = await postTo('sandbox.itunes.apple.com'); // 자동 재시도
}
if (data.status !== 0) return { verified: false, reason: `apple_status_${data.status}` };
const match = data.receipt.in_app.find(t => t.product_id === productId);

```

12.4 보안 강화 사항

- 중복 토큰 차단: `iap_receipts.purchase_token` UNIQUE 인덱스 — 같은 영수증 재사용 공격 방지
- 중복 주문 차단: `UNIQUE (store, order_id)` — 스토어가 부여한 orderId 단일성 강제
- `verified=true` 한정 지급: 검증 실패 시에도 영수증은 기록하나 wallet 변경 없음
- `raw_response` 보존: 운영 시 분쟁 대응용으로 스토어 API 원본 응답 4KB 저장
- 서버측 `entitlement`: 클라가 보낸 amount 가 아닌 서버 카탈로그 값으로 지급

Google Analytics 외부 의존 없이 서버 내장 통계로 동등한 지표를 제공한다. 클라 이벤트는 모두 `event_log` 테이블에 적재되며 어드민 대시보드에서 실시간 조회 가능.

13.1 클라 이벤트 수집

```
POST /app/member/event.json
{ userId, eventName, payload: {...}, sessionId }

⇒ event_log INSERT
⇒ members.last_active_at = NOW() (DAU 계산용)
```

주요 이벤트 (권장)

이벤트	의미	payload 예시
game_start	전투 시작	{level, opponent, mode}
game_end	전투 종료	{score, win, combo, duration}
shop_open	상점 진입	{tab}
purchase	구매 완료	{itemNo, price, currency}
skill_learn	스킬 학습	{skillId, level, point}
ad_view	광고 시청	{adId, reward}

13.2 어드민 분석 API

경로	반환
<code>/admin/analytics/overview</code>	회원수 / 신규 / DAU(오늘·어제) / WAU / 오늘 IAP
<code>/admin/analytics/dau?days=14</code>	일별 DAU 추이
<code>/admin/analytics/revenue?days=30</code>	스토어·일자별 매출 (코인 / 보석)
<code>/admin/analytics/events?hours=24</code>	이벤트명 top 50 (count + 고유 유저)
<code>/admin/analytics/ads</code>	광고 CTR (노출/클릭/% 계산)

13.3 광고 KPI 추적

노출

view_count / ad_id

클릭

click_count / ad_click_log

CTR

click / view × 100%

13.4 광고 보상 한도 (악용 방지)

- 회당 보상: 5 코인
- 1일 최대: 20회 / 100 코인
- `ad_reward_log.created_at > NOW() - 1 DAY` 카운트로 실시간 검증

별도 프론트엔드 빌드 없이 단일 HTML 파일(`public/admin.html`)로 SPA 형태의 운영 대시보드를 제공한다. `localStorage` 에 토큰을 저장하므로 새로고침 후에도 세션이 유지되며, fetch 호출에 자동 첨부된다.

14.1 8 탭 구성

탭	주요 기능	호출 API
개요	회원수 / DAU / WAU / IAP 요약 8 카드	<code>/admin/analytics/overview</code>
통계	DAU / 매출 / 이벤트 / 광고 4 표	<code>/admin/analytics/{dau,revenue,events,ads}</code>
회원	(현재 placeholder, 추후 검색)	-
결제	매출 탭 참고	<code>/admin/analytics/revenue</code>
스태이지	틀린그림 이미지 CRUD + 7좌표 검증	<code>/admin/images</code>
광고	배너 등록 / 만료 / 노출 수	<code>/admin/ads</code>
공지	제목 / 본문 / 노출 토글	<code>/admin/notices</code>
푸시	유저 단건 / 브로드캐스트 + 로그	<code>/admin/push/*</code>

14.2 인증 — X-Admin-Token

```
// Request
GET /admin/analytics/overview
X-Admin-Token: dev-admin-token-change-me // admin_tokens 테이블 조회

// Middleware (admin.js)
router.use(async (req, res, next) => {
  const token = req.headers['x-admin-token'] || req.query.adminToken;
  if (!token) return res.status(401).json({ error: 'admin_token_required' });
  const rows = await query('SELECT token FROM admin_tokens WHERE token = ? AND revoked_at IS NULL', [token]);
  if (!rows.length) return res.status(401).json({ error: 'invalid_admin_token' });
  next();
});
```

14.3 imageCut 검증 (스태이지 등록 시)

틀린그림 이미지 등록 시 7개 좌표가 이미지 경계를 벗어나는지 자동 검증.

```
function validateImageCut(cut, imgW, imgH) {
  if (!Array.isArray(cut)) return 'image_cut must be array';
  if (cut.length !== 7) return `length must be 7 (got ${cut.length})`;
  for (let i = 0; i < 7; i++) {
    const {x, y, w, h} = cut[i];
    if (![x,y,w,h].every(v => Number.isInteger(v) && v >= 0)) return `[${i}] invalid`;
    if (w === 0 || h === 0) return `[${i}] w/h must be positive`;
    if (x + w > imgW || y + h > imgH) return `[${i}] exceeds image ${imgW}x${imgH}`;
  }
  return null;
}
```

14.4 디자인 시스템

PRIMARY
#1a1f36

ACCENT
#4a90e2

SUCCESS
#27ae60

DANGER
#c0392b

15.1 FCM 푸시 알림

친구 초대 / 공지 / 보상 알림 등 다양한 시나리오. 클라이언트 오프라인 시 자동 fallback.

채널	경로	대상
토큰 등록	POST /app/member/registerFcmToken.json	클라 → 서버
유저 단건	POST /admin/push/user/:userId	운영 발송
브로드캐스트	POST /admin/push/broadcast	전체 활성 유저 (30일 이내)
발송 로그	GET /admin/push/logs	이력 조회
오프라인 초대 fallback	(소켓 107 자동 트리거)	대기실에 없는 타겟

15.2 reCAPTCHA — 가입 봇 차단

```
// /join.json + /guest.json 에 미들웨어 적용
router.all('/member/join.json', recaptcha.middleware, async (req, res) => { ... });

// recaptcha.js
async function verifyToken(token, ip) {
  const r = await fetch('https://www.google.com/recaptcha/api/siteverify', {
    method: 'POST',
    body: new URLSearchParams({ secret: process.env.RECAPTCHA_SECRET, response: token, remoteip: ip }),
  });
  const data = await r.json();
  if (data.success && data.score < MIN_SCORE) return { success: false, reason: 'score_too_low' };
  return data;
}
```

RECAPTCHA_SECRET 미설정 시 no-op — 개발/마이그레이션 환경 호환.

15.3 세션 토큰 + 중복 로그인 차단

```
// session.js
async function createSession(userId, { ip, userAgent }) {
  const token = crypto.randomBytes(24).toString('hex');
  // 기존 모든 active 세션 강제 로그아웃
  await query('UPDATE login_logs SET logged_out_at = NOW() WHERE user_id = ? AND logged_out_at IS NULL', [userId]);
  // 신규 세션 INSERT
  await query('INSERT INTO login_logs(user_id, session_token, ip, user_agent) VALUES(?, ?, ?, ?)',
    [userId, token, ip, userAgent]);
  return token;
}
```

15.4 입력 유효성 검증 — 10/10 유닛테스트 통과

필드	규칙	실패 시 코드
userId (이메일)	RFC 정규식 + 128 자 이하	email_required / email_invalid / email_too_long
userPass	4 ~ 12 자	password_length
userNick	1 ~ 20 자 (trim)	nickname_length
userCharacter	0 / 1 / 2	character_invalid

✓ 보안 체크리스트

bcrypt 해시 (cost 10) · HTTPS 강제 (cleartextTraffic=false) · UNIQUE 인덱스로 토큰 재사용 차단 · 트랜잭션 + FOR UPDATE 락 · 어드민 토큰 분리 · IAP 서버 재검증 · 광고 보상 일일 상한 · CSP 강화 (Express helmet 권장)

Google Cloud 의 Always Free 티어 + 신규 가입 \$300 크레딧을 조합해 월 \$0 운영을 목표로 한다. asia-northeast3 (서울) 은 무료 한도가 적용되지 않으므로 부득이 미국 us-central1 리전 사용 (한국 ↔ Iowa 80~120ms).

16.1 Always Free 한도 (영구 무료)

리소스	무료 한도	본 프로젝트 적용
Cloud Run	월 200만 요청 / 360,000 GB-초 메모리	REST API (예상 월 50만 요청)
Compute Engine	e2-micro 1대 (us-central1/east1/west1)	TCP 소켓 22131 + SQLite (선택)
Cloud Storage	5 GB Standard (US 리전)	이미지/사운드 에셋
Firestore	1 GB + 일 5만 read / 2만 write	(선택) MySQL 대체 후보
Cloud Build	일 120 빌드분	CI/CD
Network Egress	북미→외부 1 GB/월	API 응답 (게임 1회당 ≈ 50KB)

16.2 추천 토폴로지 — 2 옵션

OPT-A: Cloud Run + Cloud SQL (안정성)

- Cloud Run (us-central1) — REST API
- Cloud SQL db-f1-micro — MySQL (~\$9/월)
- GCE e2-micro — TCP 소켓 + Cloud SQL Proxy
- 월 비용: \$9 ~ \$12 (\$300 크레딧 25개월)
- 장점: MySQL 그대로 / 운영 안정

OPT-B: GCE + SQLite (완전 무료)

- GCE e2-micro 1대 (Always Free)
- HTTP 8080 + TCP 22131 동시 서빙
- SQLite 영구 디스크 30GB
- 월 비용: \$0 (영구)
- 장점: 무료 / 단점: 백업/복구 수동

16.3 배포 단계 (OPT-A 기준)

```
# 1. Artifact Registry
gcloud artifacts repositories create findit \
  --repository-format=docker --location=us-central1

# 2. Cloud SQL 인스턴스
gcloud sql instances create findit-db \
  --database-version=MYSQL_8_0 --tier=db-f1-micro --region=us-central1

# 3. Secret Manager
echo -n "$DB_PASSWORD" | gcloud secrets create db-password --data-file=-
echo -n "$ADMIN_TOKEN" | gcloud secrets create admin-token --data-file=-
echo -n "$RECAPTCHA" | gcloud secrets create recaptcha-key --data-file=-

# 4. Cloud Build + Cloud Run 배포
gcloud builds submit --tag us-central1-docker.pkg.dev/$PROJECT/findit/server:v1
gcloud run deploy findit-api \
  --image us-central1-docker.pkg.dev/$PROJECT/findit/server:v1 \
  --region us-central1 --allow-unauthenticated --cpu 0.5 --memory 512Mi \
  --min-instances 0 --max-instances 10 \
  --add-cloudsql-instances $PROJECT:us-central1:findit-db \
  --set-env-vars DB_SOCKET=/cloudsql/$PROJECT:us-central1:findit-db,DB_USER=findit,DB_NAME=findit_game \
  --set-secrets DB_PASSWORD=db-password:latest

# 5. APK SERVER_DOMAIN 교체
# Property.java SERVER_DOMAIN = "https://findit-api-xxxx.run.app/"
```

16.4 비용 안전장치

- 월 예산 알림 \$1 + 이메일 알림 50/90/100%
- Cloud Run min-instances=0 (콜드스타트 허용)
- Cloud Storage Standard 5GB 안에서 관리
- 일일 한도 자동 차단은 GCP 미지원 → 알림이 유일한 보호

17.1 비용 요약

항목	비용	주체	비고
GCP (Always Free + 크레딧)	\$0	교수님 카드 등록만	\$300 크레딧 90일 + 이후 Always Free
Google Play Developer	\$25 (1회)	교수님	평생 사용
Apple Developer Program	\$99/년	교수님 (또는 학교 면제)	학교 기관 면제 신청 가능
Samsung Galaxy Store	\$0	(선택)	완전 무료
도메인 (선택)	\$10~15/년	(선택)	Cloud Run 기본 URL 도 사용 가능
1년 합계 (Android+iOS)			\$124 ~ \$139

17.2 권장 출시 일정

주차	주요 작업	완료 기준
1주차	교수님 — GCP/Play 가입 + 권한 부여 / 본인 — Cloud Run 배포	Cloud Run URL HTTPS 응답
2주차	APK 서명 + 운영 URL 반영 + 내부 테스트 트랙 업로드	설치 링크 동작
3주차	내부 테스트 피드백 → 비공개 테스트 → Play 검수 신청	심사 진행
4~5주차	출시 후 모니터링 / iOS Apple Developer 가입 + TestFlight	iOS 베타 배포
6주차+	iOS 정식 출시 / 광고/공지 운영 / 패치 사이클	운영 안정화

17.3 위험 요소 + 대응

위험	영향	대응
Play/App Store 검수 거절 (개인정보)	출시 지연 1~2주	개인정보 처리방침 URL 사전 준비, 권한 최소화
\$300 크레딧 소진	월 \$9~12 청구	예산 알림 / OPT-B (GCE 무료) 전환
한국→US 리전 지연	응답 +80ms	실측 후 사용자 만족 확인, 필요 시 서울 이전 (\$35~)
FCM 키 미발급	오프라인 초대 푸시 불가	Firebase 프로젝트 생성 + 서비스 계정 등록
IAP 영수증 검증 실패율 증가	매출 누락	raw_response 분석 + Apple sandbox 자동 fallback 이미 구현
디컴파일 휴리스틱 오류	애니메이션 미세 차이	QA 시 r0/r5/r13 → mTimeCount 치환 부분 점검
한글 번역 미완 (135 줄)	해외 출시 지연	res/values-en/strings.xml 추가 작성 필요

본 프로젝트는 2014년 운영이 종료된 한국형 모바일 게임을 디컴파일·재구축하여 현대적 인프라로 다시 출시 가능한 상태로 만들었다. 원본 클라이언트 21,678 줄은 빌드 가능 상태로 복원되었고, 신규 서버 2,500 줄이 21개 REST + 7개 소켓 코드 + 어드민 대시보드를 제공한다. 기획서 17건의 모든 핵심 수치 — 44 스킬 트리, 100 레벨 능력치, 콤보 보너스, 강화 확률, 광고 한도 — 가 코드에 반영되었다.

18.1 현재 상태 요약

✓ 완료

- 서버 14 REST + 소켓 7 코드 + 16 DB 테이블
- 2013 최종 밸런스 (스킬 / 가격 / 레벨 / HP)
- 회원 시스템 (이메일 / Guest / Facebook + 세션)
- IAP 4 스토어 영수증 서버 검증
- 분석 / 광고 / 푸시 / 어드민 대시보드
- reCAPTCHA / 안정성 보장 / 로컬 테스트 통과

○ 외부 의존 (운영 시 필요)

- GCP 결제 계정 (교수님)
- Google Play / Apple Developer 가입 (교수님)
- Firebase 프로젝트 + 서비스 계정 키
- Google Play Developer API 서비스 계정
- Apple Shared Secret
- reCAPTCHA Site/Secret Key

18.2 단기 (출시 전) 추가 작업

- 클라 **Objects.java** — 서버 skill_catalog 기반 동적 스킬 생성 (현재 8개 하드코딩 → 44 확장)
- 클라 광고 배너 렌더링 — newAdImageList.json 결과를 게임 화면에 표시
- **RankActivity** — Facebook 친구 필터 토글 UI 추가
- **Facebook SDK** — Android 측 통합 + fbUserId 서버 전달
- **국제화** — res/values-en/strings.xml (135 줄 영문 번역)
- **HP 게임플레이** — 스킬이 HP 차감 / 0 시 패배 등 룰 설계 (기획서 미정)
- **KT olleh / SKT T-Store IAP 제거** — 서비스 종료된 통신사 IAP 코드 분리/제거

18.3 증장기 (운영 안정화)

- Redis 도입 (소켓 세션 분리 → 다중 인스턴스)
- Cloud CDN 적용 (이미지/사운드 가속)
- BigQuery 연동 (event_log → 장기 분석)
- Cloud Armor (DDoS 방어)
- Sentry / 알림 통합 (오류 추적)

18.4 핵심 메시지

🔴 Summary

원본 게임은 코드만 살아있고 서버가 죽은 상태였다. 본 프로젝트는 (1) 디컴파일 산출물을 빌드 가능하게 복원하고, (2) 기획서를 기반으로 신규 서버를 작성했으며, (3) GCP 무료 한도 안에서 배포 가능한 형태로 만들었다. 기술적으로는 출시 준비 완료 상태이며, 남은 것은 외부 계정 발급(교수님 카드 등록)과 스토어 심사뿐이다.